

03 — Example 1: MO-book production planning, from equations to Python objects / 示例 1：MO-book 生产计划，从公式到 Python 对象

03-E01.1 Purpose and files / 目的与文件

This learning report connects a small mathematical model to the Python objects discussed in Chapter 03. Follow one quantity from an input `float`, through a symbolic optimization expression, to a solved number and a labeled pandas table. The goal is to explain what the code means, not just obtain a solver answer.

这份学习报告把一个小数学模型与第 03 章介绍的 Python 对象联系起来。沿着一个数值的变化过程，依次认识输入 `float`、符号优化表达式、求解后的数值，以及带标签的 pandas 表格。目标是理解代码的含义，而不只是得到求解器的答案。

The example adapts MO-book §1.2, with the problem introduced in §1.1. It is a one-week, single-solve linear program (LP), using textbook data. It is neither a mixed-integer linear program (MILP) nor rolling-horizon planning, and its profit is not company performance. The adaptation preserves the five-variable model while adding numbered Spyder cells, editable inputs, result tables, and checks.

本例改编自 MO-book §1.2，题目背景见 §1.1。它使用教材数据，是单周、单次求解的线性规划（linear program, LP），既不是混合整数线性规划（MILP），也不是滚动时域计划（rolling-horizon planning）；其中的利润不代表公司经营业绩。改编保留原来的五变量模型，增加了编号的 Spyder 单元、可编辑输入、结果表格和检查。

Use the standalone [Python script](#). Its upstream copyright and MIT license are retained in [LICENSE-MO-book.txt](#). The script computes in memory and prints results; it does not read business inputs, write output files, or use a network connection.

请使用独立的 Python 脚本。上游版权声明和 MIT 许可保存在 [LICENSE-MO-book.txt](#)。脚本在内存中计算并打印结果，不读取业务输入、不写入输出文件，也不使用网络连接。

03-E01.2 Inputs, decisions, and units / 输入、决策与单位

Two products, U and V, consume material M and two kinds of labor, A and B. Choose production and the resources purchased or allocated for this week. The U sales ceiling is not a requirement to make 40 units. V has no separate sales ceiling, but labor still limits production. Money is expressed in generic monetary units. [Problem source: MO-book §1.1](#) ^[1].

两种产品 U、V 消耗原料 M，以及 A、B 两类人工。需要决定本周的产量，以及购买或安排的资源数量。U 的销量上限并不要求必须生产 40 件。V 没有单独的销量上限，但人工能力仍会限制产量。金额使用通用货币单位。题目来源：[MO-book §1.1](#) ^[1]。

Input per product unit 每件产品的输入	U	V
Selling price, monetary units/unit 售价，货币单位/件	270	210

Input per product unit 每件产品的输入	U	V
Material M, grams/unit 原料 M, 克/件	10	9
Labor A, hours/unit A 类人工, 小时/件	1	1
Labor B, hours/unit B 类人工, 小时/件	2	1
Sales ceiling, units/week 销量上限, 件/周	40	Not specified 未设置

Resource 资源	Unit cost 单位成本	Weekly availability 每周可用量
Material M 原料 M	10 monetary units/gram 10 货币单位/克	No explicit ceiling 没有显式上限
Labor A A 类人工	50 monetary units/hour 50 货币单位/小时	80 hours 80 小时
Labor B B 类人工	40 monetary units/hour 40 货币单位/小时	100 hours 100 小时

All five decisions are nonnegative and continuous. A numerical answer that happens to be an integer does not mean integer restrictions were imposed. There is no inventory carried between weeks.

五个决策变量都是非负连续变量。数值结果恰好为整数，不代表模型施加了整数约束。本例没有跨周结转的库存。

Mathematical symbol 数学符号	Code object 代码对象	Meaning and unit 含义与单位
y_U	<code>model.y_U</code>	U output, units/week U 产量, 件/周
y_V	<code>model.y_V</code>	V output, units/week V 产量, 件/周
x_M	<code>model.x_M</code>	Material allocation, grams/week 原料安排量, 克/周
x_A	<code>model.x_A</code>	Labor A allocation, hours/week A 类人工安排量, 小时/周
x_B	<code>model.x_B</code>	Labor B allocation, hours/week B 类人工安排量, 小时/周

The letters A and B name labor categories, not additional products. Check dimensions before calculating: monetary units/gram multiplied by grams/week gives monetary units/week.

字母 A、B 表示人工类别，而不是另外两种产品。计算前先核对量纲：货币单位/克乘以克/周，得到货币单位/周。

03-E01.3 The mathematical model and its implementation / 数学模型与代码实现

Maximize weekly revenue minus material and labor costs:

最大化每周收入扣除原料和人工成本后的利润：

(03-E01.1) maximize $P = 270y_U + 210y_V - 10x_M - 50x_A - 40x_B$.

Production cannot consume more resources than allocated:

生产消耗的资源不能超过已安排的资源：

(03-E01.2) $10y_U + 9y_V \leq x_M$.

(03-E01.3) $y_U + y_V \leq x_A$.

(03-E01.4) $2y_U + y_V \leq x_B$.

The variable domains and upper bounds complete the model:

再加上变量取值域与上界，模型就完整了：

(03-E01.5) $x_M \geq 0$; $0 \leq x_A \leq 80$; $0 \leq x_B \leq 100$; $0 \leq y_U \leq 40$; $y_V \geq 0$.

In S3, `pyo.NonNegativeReals` supplies nonnegativity and continuity; `bounds=(0, labor_b_limit)` supplies B's capacity.

In S4, `model.revenue` and `model.cost` are expressions, not two extra decision variables. `model.profit` is the objective with `sense=pyo.maximize`. In S5, the following exact script fragment encodes equation (03-E01.4):

在 S3 中，`pyo.NonNegativeReals` 指定非负性和连续性；`bounds=(0, labor_b_limit)` 指定 B 类人工能力上限。在 S4 中，`model.revenue` 和 `model.cost` 是表达式，不是额外增加的两个决策变量。`model.profit` 是目标，`sense=pyo.maximize` 表示最大化。下面这段与 S5 完全一致的代码实现公式 (03-E01.4)：

```
model.labor_B = pyo.Constraint(
    expr=labor_b_per_u * model.y_U + labor_b_per_v * model.y_V <= model.x_B
)
```

Here `<=` creates a symbolic relation for Pyomo; it is not a solved yes/no result. The resource coefficients map directly to `material_per_u`, `material_per_v`, `labor_a_per_u`, `labor_a_per_v`, `labor_b_per_u`, and `labor_b_per_v`. Prices map to `price_u` and `price_v`; resource cost rates map to `material_cost`, `labor_a_cost`, and `labor_b_cost`.

这里的 `<=` 为 Pyomo 创建符号关系，并不是求解后得到的“是/否”结果。资源消耗系数直接对应 `material_per_u`、`material_per_v`、`labor_a_per_u`、`labor_a_per_v`、`labor_b_per_u` 和 `labor_b_per_v`。售价对应 `price_u`、`price_v`；资源单位成本对应 `material_cost`、`labor_a_cost` 和 `labor_b_cost`。

Every resource has a positive cost. At an optimum, allocating more than production requires would waste money, so the three resource inequalities are tight. This observation permits the two-output hand calculation later; the runnable script still retains all five variables.

每种资源的成本都是正数。最优解中，如果安排的资源超过生产需要，就会浪费资金，因此三个资源不等式都取等号。利用这一点，后面可以只用两个产量进行手算；可运行脚本仍然保留全部五个变量。

03-E01.4 Read the script by stable cell number / 按固定单元编号阅读脚本

The # %% markers divide the file into Spyder cells. The cell identifiers stay meaningful even if line numbers change. Read the mathematical model alongside S2–S5, then trace the conversion into ordinary results in S6–S8.

%% 标记把文件划分成 Spyder 单元（cell）。即使代码行号改变，单元编号仍然有效。先把数学模型与 S2–S5 对照阅读，再跟踪 S6–S8 如何把模型转换为普通数值结果。

Cell 单元	Role 作用	What to inspect 重点查看
E01-S0	Scope and source 范围与来源	Textbook LP; adaptation notice 教材 LP；改编说明
E01-S1	Import libraries 导入库	pyo for modeling; pd for tables pyo 用于建模；pd 用于表格
E01-S2	Set numerical inputs 设置数值输入	Prices, costs, coefficients, capacities 售价、成本、系数、能力
E01-S3	Create model and five variables 创建模型及五个变量	Domains, bounds, unknown values 取值域、上下界、未知值
E01-S4	Define expressions and objective 定义表达式与目标	Revenue, cost, maximize profit 收入、成本、利润最大化
E01-S5	Add resource constraints 添加资源约束	Consumption versus allocation 消耗量与安排量的关系
E01-S6	Solve with HiGHS 使用 HiGHS 求解	Availability and optimal termination 求解器可用性与最优终止状态
E01-S7	Extract numbers and build tables 提取数值并建立表格	production, resources, financials
E01-S8	Recompute checks and print 复算检查并打印	checks, tolerance, displayed results checks、容差、显示结果

S6 first solves with load_solutions=False, then requires TerminationCondition.optimal before loading values. This avoids presenting unaccepted solver output as a valid solution. A missing solver or nonoptimal termination raises an error instead of silently continuing.

S6 先使用 load_solutions=False 求解，再要求终止状态为 TerminationCondition.optimal，之后才加载数值。这样可以避免把未被接受的求解器输出当作有效解。求解器不可用或未最优终止时，程序会报错，而不是无提示地继续执行。

03-E01.5 Python parameters are not optimization variables / Python 参数不等于优化变量

The word “variable” has two meanings here. Python names reference objects; an optimization decision variable represents an unknown quantity in the model. A known price may be stored under a Python name without becoming a decision for the solver.

这里的“变量”有两层含义。Python 名称引用对象；优化决策变量表示模型中的未知数量。已知售价可以保存在某个 Python 名称下，但并不会因此变成求解器需要决定的量。

Example 示例	Kind of object 对象类别	Interpretation 含义
price_u = 270.0	Python float	Known numerical input 已知数值输入
bounds=(0, labor_a_limit)	Python tuple	Pair of lower and upper bounds 由下界和上界组成的二元组
model.y_U	Pyomo variable component Pyomo 变量组件	Symbolic unknown before solving 求解前的符号未知量
model.revenue	Pyomo expression component Pyomo 表达式组件	Formula referencing decisions 引用决策变量的公式
u_units = pyo.value(model.y_U)	Solved numeric value; float in the recorded run 求解后的数值；记录的运行中为 float	Snapshot of the loaded solution 已加载解的数值快照
termination	Python str	Human-readable solver termination 可阅读的求解器终止状态
checks	pandas Series of Boolean values 包含布尔值的 pandas Series	Named validation results 带名称的验证结果

S2 uses ordinary Python floats, not `pyo.Param` components. When S3–S5 construct the model, those values enter its bounds and expressions. Reassigning `labor_b_limit = 120.0` later does not retroactively update the existing model. Rerun the whole file after input edits, including model creation and solving.

S2 使用普通 Python 浮点数，而不是 `pyo.Param` 组件。S3–S5 构建模型时，这些数值进入模型的上下界和表达式。之后重新赋值 `labor_b_limit = 120.0`，不会追溯更新已有模型。修改输入后应完整重跑文件，包括重新建模和求解。

Before solving, `model.y_U.value` is `None`: unknown, not zero. After successful solution loading, `pyo.value(model.y_U)` extracts its numerical value. The name `u_units` holds that extracted value, not a live connection that updates automatically after another solve.

求解前，`model.y_U.value` 为 `None`，表示未知，而不是零。成功加载解后，`pyo.value(model.y_U)` 提取它的数值。`u_units` 保存的是提取出来的值，不是再次求解后会自动更新的实时连接。

For inspection, run the following after the complete script. Type names for library components can be more specific than the broad categories above; that is normal.

完整运行脚本后，可以执行以下语句进行查看。库组件的实际类型名称可能比上表中的类别更具体，这是正常现象。

```
type(price_u)
type(model.y_U)
type(u_units)
model.y_U.value
model.labor_B.pprint()
```

03-E01.6 From dictionary keys to pandas labels / 从字典键到 pandas 标签

In this exact S7 fragment, braces create a Python dictionary. Its string keys become the Series index labels; the numeric values become the Series contents. The `name` describes the whole Series, not another entry.

下面这段与 S7 完全一致的代码中，花括号创建 Python 字典（dict）。字符串键（key）成为 Series 的索引标签，数值成为 Series 的内容。`name` 描述整个 Series，而不是新增一个条目。

```
financials = pd.Series({
    "revenue": revenue_value, "cost": cost_value, "profit": profit_value,
}, name="monetary_units_per_week")
```

A Series is one-dimensional labeled data. `financials.loc["profit"]` selects by label; `financials.iloc[2]` selects the third position, counting from zero. They coincide here because of the construction order, but position is not the meaning of the label. Prefer the label when asking for profit. See the [Series API](#) ^[2] and [indexing guide](#) ^[3].

Series 是带标签的一维数据。`financials.loc["profit"]` 按标签选择；`financials.iloc[2]` 按从零开始计数的位置选择第三项。本例中两者因构造顺序而得到相同结果，但位置并不是标签的含义。想读取利润时，优先使用利润标签。参见 [Series API](#) ^[2] 和 [索引指南](#) ^[3]。

The second exact S7 fragment uses a dictionary differently: its keys become DataFrame column labels and its equal-length lists supply the columns.

第二段与 S7 完全一致的代码以另一种方式使用字典：键成为 DataFrame 的列标签，等长的列表提供各列的数据。

```
production = pd.DataFrame({
    "product": ["U", "V"],
    "quantity_units_per_week": [u_units, v_units],
})
```

A DataFrame is two-dimensional. Here the default row index is 0, 1; U and V are values in the product column, not row labels. Thus `production.loc[0, "quantity_units_per_week"]` selects row label 0 and the named column, whereas `production.iloc[0, 1]` selects the first row and second column by position. `production["product"]` returns a Series. See the [DataFrame API](#) ^[4].

DataFrame 是二维数据。这里默认的行索引为 0, 1; U、V 是 product 列中的值，而不是行标签。因此，`production.loc[0, "quantity_units_per_week"]` 选择行标签 0 和指定名称的列，而 `production.iloc[0, 1]` 按位置选择第一行、第二列。`production["product"]` 返回一个 Series。参见 [DataFrame API](#) ^[4]。

Python dictionary keys are unique: assigning the same key again replaces its associated value. pandas labels are not universally required to be unique. A dictionary's key rule therefore must not be mistaken for a general Series or DataFrame restriction. The containers are related by construction, not identical types. [Python dictionary reference](#) ^[5].

Python 字典的键具有唯一性：再次对同一键赋值，会替换对应值。pandas 标签则并非一律要求唯一。因此，不能把字典的键规则误当作 Series 或 DataFrame 的普遍限制。这些容器在构造方式上有关联，但不是同一种类型。参见 [Python 字典参考](#) ^[5]。

The resources DataFrame similarly uses the keys resource, allocated, limit, and unit as columns. Its limit list contains [None, labor_a_limit, labor_b_limit]; the material row displays NaN in this numeric column. Here that intentionally represents “no explicit material ceiling,” not zero capacity. This is a modeling convention, not a universal interpretation of missing data.

resources DataFrame 同样把 resource、allocated、limit、unit 这些键作为列名。其 limit 列表是 [None, labor_a_limit, labor_b_limit]; 原料行在这一数值列中显示为 NaN。这里有意用它表示“没有显式原料上限”，而不是零能力。这是本例的建模约定，不是缺失数据的通用解释。

03-E01.7 Run and inspect in Spyder / 在 Spyder 中运行与查看

Use the existing Jushi_scheduler environment with pyomo, highspy, and pandas. In an Anaconda Prompt, start from the handbook root:

使用已有的 Jushi_scheduler 环境，其中需要 pyomo、highspy 和 pandas。在 Anaconda Prompt 中，从手册根目录运行：

```
conda activate Jushi_scheduler
python examples\03_example_01_mo_book_production.py
```

For Spyder, use this procedure:

在 Spyder 中按以下步骤操作：

1. Open the same script and use a console whose Python interpreter belongs to Jushi_scheduler. Check `import sys; print(sys.executable)` in that console if uncertain.
打开同一个脚本，使用 Python 解释器（interpreter）属于 Jushi_scheduler 的控制台。如果不确定，在该控制台执行 `import sys; print(sys.executable)` 检查。
2. Run the entire file first. Read the termination message and confirm that the independent checks pass.
先完整运行文件。阅读终止状态消息，确认独立复算检查通过。
3. In Variable Explorer, open production, resources, financials, and checks. These ordinary tables are easier to inspect than the nested model object.
在变量浏览器（Variable Explorer）中打开 production、resources、financials、checks。这些普通表格比层层嵌套的 model 对象更容易查看。
4. To study construction, restart the console and run cells in order from E01-S1 through E01-S8. Before S6, inspect `model.y_U.value`; after S7, inspect `u_units`.
为了学习构建过程，重启控制台，按顺序运行 E01-S1 至 E01-S8。S6 之前查看 `model.y_U.value`；S7 之后查看 `u_units`。
5. After changing S2 inputs, run the entire file again. Do not mix tables from an old solution with a newly edited parameter.
修改 S2 输入后，再次完整运行文件。不要把旧解的表格与刚修改的参数混在一起解释。

The console can also run `model.pprint()` to display model structure. If HiGHS is unavailable or an import fails, check the active interpreter first; installing packages into a different environment will not repair this console. Spyder interface labels and shortcuts may vary, so use its run-file and run-cell commands. [Spyder editor documentation](#) ^[6].

也可以在控制台执行 `model.pprint()` 显示模型结构。如果 HiGHS 不可用或导入失败，先检查当前解释器；把包安装到另一个环境中，并不能修复这个控制台。Spyder 的界面名称和快捷键可能不同，请使用对应的“运行文件”和“运行单元”命令。参见 [Spyder 编辑器文档](#) ^[6]。

03-E01.8 Results, feasibility, and an optimality certificate / 结果、可行性与最优性证明

For the baseline inputs, reproduce the following result:

使用基准输入，应复现以下结果：

Output 输出	Value 数值
U production, units/week U 产量，件/周	20
V production, units/week V 产量，件/周	60
Material, grams/week 原料，克/周	740
Labor A, hours/week A 类人工，小时/周	80
Labor B, hours/week B 类人工，小时/周	100
Revenue, monetary units/week 收入，货币单位/周	18000
Cost, monetary units/week 成本，货币单位/周	15400
Profit, monetary units/week 利润，货币单位/周	2600

Hand checks make the answer interpretable: material is $10 \times 20 + 9 \times 60 = 740$; labor A is $20 + 60 = 80$; labor B is $2 \times 20 + 60 = 100$. Revenue is $270 \times 20 + 210 \times 60 = 18000$, and cost is $10 \times 740 + 50 \times 80 + 40 \times 100 = 15400$. U has 20 units of unused sales allowance; both labor capacities are fully used.

手算有助于解释结果：原料为 $10 \times 20 + 9 \times 60 = 740$ ；A 类人工为 $20 + 60 = 80$ ；B 类人工为 $2 \times 20 + 60 = 100$ 。收入为 $270 \times 20 + 210 \times 60 = 18000$ ，成本为 $10 \times 740 + 50 \times 80 + 40 \times 100 = 15400$ 。U 距离销量上限还剩 20 件；两类人工能力都已用满。

The eight S8 checks test nonnegativity, three upper bounds, three resource balances, and the profit identity, with tolerance = 1e-6 for floating-point noise. Passing these checks establishes the tested feasibility and consistency conditions; primal feasibility alone does not prove optimality. The solver termination check is separate.

S8 的八项检查涵盖非负性、三个上界、三个资源平衡关系和利润恒等式，并用 tolerance = 1e-6 处理浮点误差。通过这些检查，说明所检查的可行性和一致性条件成立；仅有原始可行性（primal feasibility），不能证明最优性。求解器终止状态是另行检查的。

There is also a short analytical certificate. Write $U = y_U$ and $V = y_V$. With no resource waste, each U contributes $270 - 100 - 50 - 80 = 40$, and each V contributes $210 - 90 - 50 - 40 = 30$. Any excess allocation only lowers profit. Therefore every feasible plan satisfies:

还可以给出一个简短的解析最优性证明。记 $U = y_U$ ， $V = y_V$ 。不浪费资源时，每件 U 的贡献为 $270 - 100 - 50 - 80 = 40$ ，每件 V 的贡献为 $210 - 90 - 50 - 40 = 30$ 。多安排资源只会降低利润。因此，每个可行方案都满足：

(03-E01.6) $P \leq 40U + 30V = 20(U + V) + 10(2U + V) \leq 20 \times 80 + 10 \times 100 = 2600$.

The reported plan is feasible and attains 2600, so no feasible plan can improve it. This is an optimality argument, not just substitution into constraints. It uses the baseline coefficients and must be reconsidered after changing inputs.

报告中的方案可行，而且达到 2600，因此不存在利润更高的可行方案。这是在论证最优性，而不只是把数值代入约束。该证明使用基准系数；改变输入后，需要重新审视证明。

03-E01.9 Controlled experiments and next steps / 单因素实验与后续学习

Change one input at a time, rerun all cells, record the result, and restore the baseline before the next experiment.

每次只改变一个输入，完整重跑所有单元，记录结果，并在下一个实验前恢复基准输入。

1. Increase `labor_b_limit` from 100 to 120. Expect $U = 40$, $V = 40$, and profit = 2800. Explain why greater B capacity changes the product mix although A capacity stays at 80.
把 `labor_b_limit` 从 100 增加到 120。预期 $U = 40$ 、 $V = 40$ 、利润 = 2800。解释为什么 A 类能力仍为 80，但增加 B 类能力会改变产品组合。
2. Set `labor_a_limit` to zero with other inputs restored. Expect zero production, resource allocation, and profit: both products require A labor.
恢复其他输入后，把 `labor_a_limit` 设为零。预期产量、资源安排量和利润均为零，因为两种产品都需要 A 类人工。
3. Retrieve profit using `.loc`, and retrieve V output from the DataFrame. Explain why `.loc["V"]` does not work on production as constructed.
使用 `.loc` 读取利润，再从 DataFrame 读取 V 的产量。解释为什么按照当前构造方式，不能对 `production` 直接使用 `.loc["V"]`。
4. Explain why `type(model.y_U)` differs from `type(u_units)`, and why an integer-looking production result is still an LP solution.
解释为什么 `type(model.y_U)` 与 `type(u_units)` 不同，以及为什么看起来是整数的产量结果仍然是 LP 的解。

The script does not implement integer batches, setup decisions, equipment assignment, due dates, inventory dynamics, or uncertainty. A next example could introduce explicit integer/binary decisions and inventory balances to build a MILP. A subsequent rolling-horizon example would repeatedly solve a model, implement only the current decisions, and transfer the resulting state to the next window. Those are proposed learning steps, not features already implemented here.

脚本没有实现整数批次、开停或换产决策、设备分配、交期、库存动态或不确定性。下一个示例可以引入显式的整数/二元决策和库存平衡，建立 MILP。之后的滚动时域示例可以重复求解模型，只执行当前阶段的决策，并把执行后的状态传给下一窗口。这些是建议的后续学习步骤，不是本例已经实现的功能。

03-E01.10 Reproducibility and sources / 可复现性与来源

The accompanying [verification record](#) identifies the tested script by a SHA-256 content hash and records the actual checks. The recorded environment is Python 3.12.13, Pyomo 6.10.1, highspy 1.15.1, and pandas 3.0.5 in `Jushi_scheduler`, dated 2026-09-25. Consult that record when comparing results after edits; an old hash does not verify changed code.

配套的验证记录用 SHA-256 内容哈希标识被测试的脚本，并记录实际检查。记录的环境为 `Jushi_scheduler` 中的 Python 3.12.13、Pyomo 6.10.1、highspy 1.15.1 和 pandas 3.0.5，日期为 2026-09-25。修改代码后比较结果时，应查阅该记录；旧哈希不能验证已经修改的代码。

The separate [verification script](#) can be run from the handbook root in the same environment:

独立的验证脚本 可以在同一环境中，从手册根目录运行：

```
python examples\verify_03_example_01.py
```

Five cases passed on 2026-09-25: the baseline, B capacity 120, A capacity zero, all nine cells executed in order in a shared namespace, and deliberately infeasible A capacity -1. The last case raised an error before loading a solution: there were zero solution-load calls, and all five decision values remained unset. The negative-capacity case tests failure handling; it is not a sensible business scenario.

2026-09-25 验证通过了五种情况：基准输入、B 类能力为 120、A 类能力为零、九个单元在共享命名空间中按序执行，以及故意设置不可行的 A 类能力 -1。最后一种情况在加载解之前报错：加载解的调用次数为零，五个决策变量的值都保持未设置状态。负能力案例用于测试失败处理，并不是合理的业务情景。

Command-line execution and shared-namespace cell execution are distinct from operating Spyder's graphical interface. No Spyder GUI run or performance benchmark is claimed. The small tests check this teaching artifact, not a factory scheduling system or complete business-input validation.

命令行执行和共享命名空间下的逐单元执行，不等于操作 Spyder 图形界面。本报告不声称进行了 Spyder GUI 运行或性能基准测试。这些小测试检查的是教学材料本身，而不是工厂排产系统，也不是完整的业务输入合法性验证。

The problem and modeling starting point are [MO-book §1.1](#) ^[1] and [MO-book §1.2](#) ^[7]. Code attribution is retained under the [upstream MIT license](#) ^[8]. For the library concepts, use the linked Python and pandas references above and [Pyomo's modeling components documentation](#) ^[9]. The cell numbering, object tracing, result checks, and hand certificate in this report explain the local learning adaptation.

题目和建模起点分别是 [MO-book §1.1](#) ^[1] 和 [MO-book §1.2](#) ^[7]。代码按照上游 MIT 许可 ^[8] 保留署名。关于库的概念，可查阅上文链接的 Python、pandas 参考资料，以及 [Pyomo 建模组件文档](#) ^[9]。本报告中的单元编号、对象跟踪、结果检查与手算证明，用于解释本地学习改编版。

参考资料 (References)

1. <https://mobook.github.io/MO-book/notebooks/01/01-production-planning.html>
2. <https://pandas.pydata.org/docs/reference/api/pandas.Series.html>
3. https://pandas.pydata.org/docs/user_guide/indexing.html
4. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>
5. <https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>
6. <https://docs.spyder-ide.org/current/panes/editor.html>
7. <https://mobook.github.io/MO-book/notebooks/01/02-production-planning-basic.html>
8. <https://github.com/mobook/MO-book/blob/main/LICENSE>
9. https://pyomo.readthedocs.io/en/stable/explanation/modeling/math_programming/index.html